

TABLE OF CONTENTS

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	CERTIFICATE	3
	DECLARATION	5
	ACKNOWLEDGEMENT	7
1	INTRODUCTION	12
1.1	Synopsis	13
1.2	About the project	14
1.3	Introduction	15
1.4	Data Overview	16
2.	DATA PREPROCESSING	17
2.1	Data Cleaning	18
2.2	Data Conversion	20
2.3	Text Data Pre-processing	21
2.4	Data Aggregation	22
2.5	Data Splitting	23
2.6	Normalization and Scaling	24
3	DATA ANALYSIS & DATA VISUALIZATION	25
3.1	Transaction Pattern Analysis	26
3.2	Fraud Risk Analysis	34

3.3	Extra Insights Analysis	43
3.4	Geo Fraud Map (India)	40
4	DATA MODELLING	48
4.1	Predictive Fraud Model	49
3.5	Correlation Matrix	
5	CONCLUSION	53
5.1	Conclusion	54
5.2	Future Enhancement	55
5.3	References	56

INTRODUCTION

1.1 SYNOPSIS

The unified payments interface (UPI) has revolutionized INDIA'S digital payment ecosystem by enabling instant and secure financial transactions. However, with this growing convenience, the risk of fraudulent activities has also increased. This project focuses on building a fraud detection system that uses data science and machine learning techniques to identify suspicious upi transactions effectively.

The dataset contains transaction details such as transaction ID, timestamp, amount, device type, merchant category, and transaction status. The project workflow includes data cleaning, data conversion, feature engineering, text data preprocessing, data aggregation, and visualization. Fraudulent transactions are labeled using rule-based conditions such as abnormal time of transaction, unusually high amounts, and failed or declined statuses.

Exploratory Data Analysis (EDA) has been carried out to visualize hidden trends using histograms, box plots, and heatmaps. These visual insights have guided the feature selection and model design process. A Random Forest Classifier has been applied for prediction, and the SMOTE technique has been used to handle the imbalance between legitimate and fraudulent cases. Evaluation metrics such as accuracy, precision, recall, and F1-score have been used to measure performance.

This project demonstrates that combining data-driven modeling with rule-based detection improves the identification of fraudulent transactions, thus contributing to safer digital payment systems.

1.2 ABOUT THE PROJECT

The main objective of this project is to develop an automated fraud detection model for UPI transactions. With the increasing reliance on digital payments, even a small percentage of fraudulent activities can cause significant losses. Therefore, an intelligent and scalable system capable of detecting fraudulent patterns is essential.

The dataset used consists of around 60,000 transaction records, each containing important fields such as transaction amount, device type, timestamp, and transaction status. Preprocessing steps were implemented to remove duplicates, handle missing values, and standardize formats. A new variable, `fraud_flag`, was created to indicate potentially fraudulent transactions based on defined rules.

After preprocessing, Exploratory Data Analysis (EDA) revealed meaningful insights such as higher fraud rates in transactions during late-night hours and among certain merchant categories. The Random Forest Classifier was chosen for its robustness in handling both numerical and categorical data. SMOTE balancing ensured that minority (fraud) cases were sufficiently represented during training.

The final model showed strong predictive performance and reliability. Overall, the project provides a comprehensive framework combining data preprocessing, visualization, and machine learning for efficient fraud detection in UPI-based payment systems.

1.3 INTRODUCTION

The rise of digital payments has significantly changed how financial transactions are conducted in India. Among these platforms, the Unified Payments Interface (UPI) has become the most widely adopted system for instant and secure money transfers. However, this growing popularity has also made UPI a target for cybercriminals, leading to various forms of fraudulent activities.

Fraud detection in such systems is a critical task because traditional rule-based systems alone are insufficient to capture complex or evolving fraud patterns. Hence, this project uses a hybrid approach that combines rule-based identification with machine learning prediction to detect fraudulent UPI transactions.

The project includes multiple stages:

1. Data Preprocessing: Cleaning and structuring data for analysis.
2. Exploratory Data Analysis (EDA): Understanding relationships between variables and identifying potential risk factors.
3. Feature Engineering: Extracting time-based and categorical features to capture transaction behavior.
4. Model Building: Applying Random Forest Classifier to predict fraud.
5. Evaluation: Using metrics such as accuracy, precision, and recall to assess model effectiveness.

This combined approach ensures that both known and hidden fraud patterns can be identified. The outcome is a reliable and adaptable model that enhances the safety of digital payment systems.

1.4 DATA OVERVIEW

The dataset used in this project is titled “**UPI Enriched Transaction Dataset**”, which contains detailed information on digital payment transactions performed through the Unified Payments Interface (UPI). It comprises approximately **60,000 records**, with each row representing a unique UPI transaction made by individual users or merchants across India.

The dataset has been designed to capture multiple aspects of a transaction such as the **sender, receiver, transaction details, device type, location, and transaction outcome**. These attributes enable both analytical exploration and predictive modeling to identify potential fraudulent activities.

Column Name	Description
transaction_id	Unique identifier assigned to each UPI transaction.
sender_name	Masked name or ID of the user initiating the transaction.
receiver_name	Masked name or ID of the user receiving the payment.
sender_bank	Bank associated with the sender’s UPI account.
receiver_bank	Bank associated with the receiver’s UPI account.
amount_inr	Monetary value of the transaction in Indian Rupees (INR).
transaction_type	Indicates whether the transaction is P2P (Person-to-Person) or P2M (Person-to-Merchant) .
transaction_status	Represents the outcome of the transaction, such as Success, Failed, Pending, or Suspicious .
device_type	Type of device used for the transaction, e.g., Android, iOS, or Desktop .

merchant_category	Business category of the receiving merchant (e.g., Retail, Utilities, Food, Travel, etc.).
geo_region	Geographical region where the transaction took place.
state	Indian state extracted from the user's location data (used for Geo Fraud Map).
latitude	Latitude coordinate of the transaction location.
longitude	Longitude coordinate of the transaction location.
sender_age_group	Categorical representation of the sender's age group (e.g., 18–25, 26–35, etc.).
account_tenure	Duration (in months or years) that the account has been active.
time_stamp	Timestamp indicating the exact date and time of the transaction.
hour	Derived feature representing the hour of the day (used for time-based analysis).
day	Derived feature representing the day of the week (e.g., Monday, Tuesday, etc.).
month	Derived feature representing the month of the transaction.
device_type_encoded	Numerical encoding of the device_type column for modeling.
status_encoded	Numerical encoding of the transaction_status column.

IMPORTING LIBRARIES

1. pandas

- **Purpose:** Used for data manipulation, cleaning, and analysis of structured data.
- **Role:** Loaded the `upi_enriched.csv` dataset, removed duplicates, handled missing values, and created new columns such as *hour*, *day*, and *month* for fraud pattern analysis.

2. numpy

- **Purpose:** Provides support for fast numerical computations and mathematical operations.
- **Role:** Assisted in numerical processing, conversions, and efficient handling of large numerical arrays during feature engineering.

3. matplotlib.pyplot

- **Purpose:** Used for creating basic static visualizations like line charts, histograms, and bar plots.
- **Role:** Helped visualize transaction patterns such as hourly transaction volume and fraud distribution across categories.

4. seaborn

- **Purpose:** An advanced visualization library for statistical data representation.
- **Role:** Created detailed visualizations such as count plots, scatter plots, and heatmaps to analyze transaction type distribution, device usage, and merchant fraud rates.

5. LabelEncoder (from sklearn.preprocessing)

- **Purpose:** Converts categorical data into numerical values for machine learning.
- **Role:** Transformed categorical columns like *device_type* and *transaction_status* into numeric form for model training.

6. train_test_split (from sklearn.model_selection)

- **Purpose:** Splits the dataset into training and testing sets for model evaluation.

- **Role:** Divided the data in a 70:30 ratio to train and test the Random Forest Classifier effectively.

7. RandomForestClassifier (from sklearn.ensemble)

- **Purpose:** A supervised ensemble learning algorithm used for classification problems.
- **Role:** Served as the main predictive model to detect fraudulent transactions based on transaction features.

8. confusion_matrix & classification_report (from sklearn.metrics)

- **Purpose:** Evaluate model accuracy and performance metrics.
- **Role:** Used to calculate precision, recall, F1-score, and visualize true vs. false predictions in the fraud detection model.

9. SMOTE (from imblearn.over_sampling)

- **Purpose:** Handles class imbalance by generating synthetic samples of minority classes.
- **Role:** Balanced the dataset by oversampling the fraud (minority) class to improve Random Forest performance.

10. geopandas

- **Purpose:** Extends pandas for handling geospatial data and creating maps.
- **Role:** Used to plot the **Geo Fraud Map of India**, showing fraud rate variations across states using `india_state.geojson`.

11. re (Regular Expressions)

- **Purpose:** Provides pattern matching and text cleaning capabilities.
- **Role:** Cleaned textual data in columns like *merchant_category* by removing numbers, punctuation, and unwanted characters.

12. nltk.corpus.stopwords

- **Purpose:** Supplies a list of common words (stopwords) to remove during text preprocessing.
- **Role:** Eliminated frequent but unimportant words (e.g., “the”, “and”, “is”) from text fields to improve preprocessing quality.

13. nltk.stem.PorterStemmer

- **Purpose:** Reduces words to their root or base form (stemming).

- **Role:** Standardized text by converting words like “payments”, “paying”, and “payment” into their common base form “payment”.

14. nltk.download('stopwords')

- **Purpose:** Downloads the NLTK stopword list for text cleaning.
- **Role:** Ensured that all text preprocessing operations could run smoothly by making the stopwords corpus available

```
# =====
# ----- Import Libraries -----
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report
from imblearn.over_sampling import SMOTE
import geopandas as gpd
import re
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
import nltk
nltk.download('stopwords')
```

DATA LOADING

The dataset used for this project, named “upi_enriched.csv”, was loaded using the pandas library. This dataset contains approximately 60,000 UPI transaction records, each representing an individual digital payment made through the Unified Payments Interface (UPI). The data was imported into a pandas DataFrame to allow easy manipulation and analysis. Once loaded, the structure of the dataset, including the number of rows, columns, and data types, was verified. This step ensured that the dataset was successfully read and ready for preprocessing, analysis, and model building.

CODE:

```
..  
df = pd.read_csv("/content/upi_enriched.csv")
```

DATA PREPROCESSING

2 DATA PREPROCESSING

Data preprocessing was carried out to clean and prepare the dataset for analysis and modeling. Duplicate records were removed, missing values were handled, and column names were standardized. Text data such as merchant categories were cleaned using regular expressions and processed with stopword removal and stemming. Categorical fields were encoded using Label Encoding, and time-based features like hour, day, and month were extracted. This ensured the dataset was accurate, consistent, and ready for fraud detection.

2.1 DATA CLEANING

The data cleaning process has been carried out as the first step to ensure that the dataset is accurate, consistent, and ready for further analysis. Since the dataset contains more than 60,000 UPI transaction records, a few common issues such as duplicate entries and missing values have been handled carefully to make the dataset reliable.

Firstly, load the dataset, **cleaning column and duplicate records have been identified and removed**. Duplicate transactions can mislead the analysis by repeating the same record multiple times, which affects the accuracy of transaction counts and the overall fraud pattern analysis. Removing duplicates ensures that every transaction is unique.

CODE:

```
# clean the column
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_").
                                                    str.replace("(", "").str.replace(")", "")
df.drop_duplicates(inplace=True)
```

Secondly, **missing values** have been handled. Certain records in the dataset had missing entries for important columns such as transaction amount, transaction status, and device_type. Transactions with missing critical data have been dropped, while less critical missing categorical values have been filled with the default label "Unknown". This ensures that the dataset remains consistent and that no null entries affect future computations or visualizations.

CODE:

```
# filling missing value
df.dropna(subset=['amount_inr', 'transaction_status', 'device_type'], inplace=True)
df['transaction_status'] = df['transaction_status'].fillna('Unknown')
```

Thirdly, **label encoding** has been applied to convert the text-based categorical columns into numerical values.

This step allows machine learning algorithms to understand and process the categorical features, such as device type and transaction status.

CODE:

```
# --- Label Encoding ---
label_enc = LabelEncoder()
df['device_type_encoded'] = label_enc.fit_transform(df['device_type'].astype(str))

df['status_encoded'] = label_enc.fit_transform(df['transaction_status'].astype(str))
```


2.2 DATA CONVERSION

The data conversion step has been applied to transform attributes into suitable datatypes for analysis. Initially, the dataset has contained fields in mixed formats, such as timestamps stored as plain text and amounts stored as strings.

The transaction timestamp has been converted into a proper datetime format. This conversion has allowed new features such as transaction hour, day, and month to be derived easily. Time-based analysis has played a major Role in identifying unusual behavior, such as transactions that have taken place during late-night hours.

Similarly, transaction amounts have been converted into numeric values. In some cases, the column has contained non-numeric characters or invalid entries, which have been coerced into missing values and handled during cleaning. Once converted, the amounts have been available for calculations, aggregations, and modeling.

These conversions have ensured that every field has been represented in the most meaningful way, which has enhanced the efficiency and accuracy of subsequent steps.

CODE:

```
df['amount_inr'] = pd.to_numeric(df['amount_inr'], errors='coerce')  
df['time_stamp'] = pd.to_datetime(df['time_stamp'], errors='coerce')
```

2.3 FEATURE ENGINEERING

Feature engineering was performed to create new meaningful variables that help improve fraud detection accuracy. From the transaction timestamp, new features such as **hour**, **day**, and **month** were extracted to analyze time-based transaction patterns. A binary variable called **fraud_flag** was created to label suspicious transactions based on rules like high transaction amounts, failed status, or unusual transaction times. These engineered features helped reveal hidden patterns in the data and enhanced the performance of the predictive model.

CODE:

```
# --- Feature Engineering ---  
df['hour'] = df['time_stamp'].dt.hour  
df['day'] = df['time_stamp'].dt.day_name()  
df['month'] = df['time_stamp'].dt.month_name()
```

2.3 TEXT DATA PRE-PROCESSING

Text preprocessing is performed to clean and standardize textual data such as *merchant categories* and *transaction types* before analysis. It helps remove noise and prepare the text for encoding or feature extraction. Steps involved:

- Lowercasing: Converts all text to lowercase to maintain uniformity.
- Removing punctuation and special characters: Filters out unnecessary symbols that add no analytical value.
- Tokenization and stopwords removal: Breaks text into individual words and removes commonly used words like “*the*”, “*is*”, etc.
- Stemming: Reduces words to their root form (e.g., “*payments*” → “*payment*”) using algorithms such as the Porter Stemmer.

This ensures that textual attributes are consistent, compact, and easier for machine learning algorithms to interpret.

CODE:

```
ps = PorterStemmer()
stop_words = set(stopwords.words('english'))
def clean_text(text):
    if pd.isna(text):
        return ""
    text = re.sub(r'^a-zA-Z\s|', "", str(text)).lower()
    return ' '.join([ps.stem(word) for word in text.split() if word not in stop_words])

for col in ['merchant_category', 'transaction_type']:
    if col in df.columns:
        df[col] = df[col].apply(clean_text)
```

2.4 DATA AGGREGATION

Data aggregation combines and summarizes transaction-level data to generate higher-level insights.

For this project, transactions were aggregated by sender_name to compute:

- Transaction Count: Total number of transactions per user.
- Average Transaction Amount: Mean value of transactions to understand spending behavior.
- Fraud Count: Total number of fraudulent transactions per user.

These aggregated metrics help in identifying customer behavior patterns and assessing fraud risk at the user or merchant level.

CODE:

```
agg_data = df.groupby('sender_name').agg({
    'transaction_id': 'count',
    'amount_inr': 'mean',
    'fraud': 'sum'
}).reset_index().rename(columns={
    'transaction_id': 'transaction_count',
    'amount_inr': 'avg_amount',
    'fraud': 'fraud_count'
})
print("\nAggregated Data Sample:\n", agg_data.head())
```

OUTPUT:

```
Aggregated Data Sample:
   sender_name  transaction_count  avg_amount  fraud_count
0  Amit Sharma                5549   1303.000505   3511.563541
1  Anjali Gupta                5629   1333.455135   3550.849633
2   Kavya Nair                5547   1352.002524   3519.977209
3  Manoj Yadav                5670   1309.332669   3554.777955
4   Neha Sharma                5418   1274.095295   3429.217541
```

2.5 DATA SPLITTING

Data splitting separates the dataset into training and testing subsets for unbiased model evaluation.

- Training Set (70%) is used to teach the model patterns within the data.
- Testing Set (30%) is reserved to evaluate model accuracy and performance on unseen data.

A stratified split is used to preserve the proportion of fraud and non-fraud cases in both sets, ensuring a balanced representation and preventing bias in model learning.

CODE:

```
# Re-define X and y
X = df[['amount_inr','device_type_encoded','status_encoded']]
y = df['fraud']

# Handle classes with only one member in the target variable
class_counts = y.value_counts()
classes_to_remove = class_counts[class_counts < 2].index.tolist()

if classes_to_remove:
    print(f"Warning: Removing classes with less than 2 members from target variable:
    {classes_to_remove}")
    indices_to_keep = y[~y.isin(classes_to_remove)].index
    X = X.loc[indices_to_keep]
    y = y.loc[indices_to_keep]
else:
    print("\nSkipping data splitting and scaling: X or y is not valid, or not enough
    classes with >= 2 members after removing rare classes.")
```

2.6 NORMALIZATION AND SCALING

Normalization and scaling are used to bring all numerical features to a similar range, preventing attributes with large magnitudes from dominating model training. Two popular methods are used:

1. Standard Scaling (Z-score):
 - Centers features around a mean of 0 with a standard deviation of 1.
 - Best suited for models assuming normally distributed data (e.g., Random Forest, Logistic Regression).
2. Min-Max Scaling:
 - Rescales values to a fixed range [0, 1].
 - Useful for distance-based algorithms or when different features have varying units.

Applying scaling ensures faster convergence, stable training, and balanced contribution of all features in fraud detection.

CODE:

```
if not X.empty and len(y) == len(X) and all(y.value_counts() >= 2):  
    # Data Splitting (Stratified)  
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,  
random_state=42, stratify=y)  
  
    # Normalization and Scaling  
    scaler = StandardScaler()  
    X_train_scaled = scaler.fit_transform(X_train)  
    X_test_scaled = scaler.transform(X_test)  
    print("\nData normalized and scaled successfully!")
```

2.7 FRAUD FLAG CREATION

A **fraud flag** was created to identify and label suspicious transactions in the dataset. Since the dataset did not include a direct fraud indicator, rule-based conditions were applied to generate this label. Transactions were marked as **fraudulent (1)** if they showed abnormal behavior such as **failed or suspicious status**, **unusually high transaction amounts (above ₹2,00,000)**, or **transactions made at odd hours**. All other transactions were labeled as **non-fraudulent (0)**. This binary fraud flag served as the **target variable** for the predictive model, allowing the system to learn and classify future transactions effectively.

CODE:

```
# =====  
# ----- Fraud Flag Creation -----  
# =====  
df['fraud'] = 0  
df.loc[df['transaction_status'].isin(['FAILED', 'SUSPICIOUS']), 'fraud'] = 1  
df.loc[df['amount_inr'] > 200000, 'fraud'] = 1  
if 'account_tenure' in df.columns:  
    df.loc[(df['account_tenure'] < 30) & (df['amount_inr'] > 50000), 'fraud'] = 1  
  
print("Fraud cases flagged:", df['fraud'].sum())  
print("Available columns after cleaning:", df.columns.tolist())
```

OUTPUT:

```
Fraud cases flagged: 28371  
Available columns after cleaning: ['transaction_id', 'time_stamp', 'transaction_type', 'merchant_category',
```

**DATA ANALYSIS
& DATA VISUALIZATION**

3.0 DATA ANALYSIS & DATA VISUALIZATION

Data analysis was carried out to explore the structure and patterns within the UPI transaction dataset. Various attributes such as transaction amount, type, device used, and merchant category were examined to understand user behavior and identify irregular activities. The analysis helped in detecting trends like high-value transactions, failed payments, and frequent users, which are potential indicators of fraud. This stage provided the foundation for selecting the most relevant features for modeling and further visualization.

Data visualization was performed to represent transaction patterns and fraud risks in a clear and understandable way. Different plots such as count plots, bar charts, histograms, and line graphs were created using *Matplotlib* and *Seaborn*. These visualizations illustrated transaction trends over time, device preferences, regional distribution, and merchant risk categories. By visually analyzing the data, hidden relationships were uncovered, making it easier to interpret and explain the factors contributing to fraudulent activities.

3.1 TRANSACTION PATTERN ANALYSIS

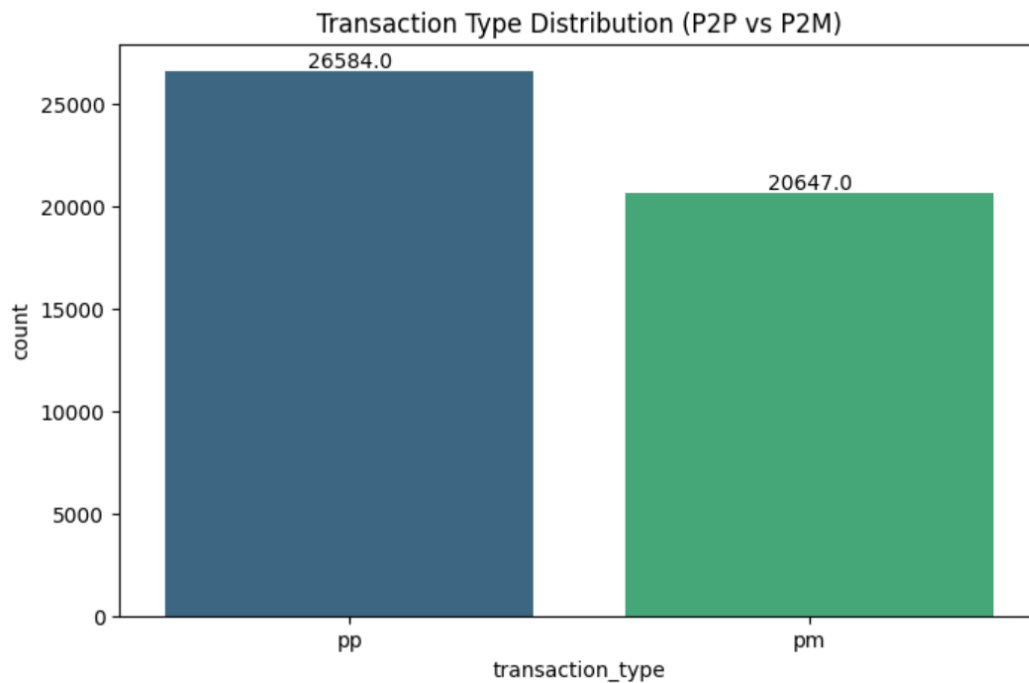
3.1.1 TRANSACTION TYPE DISTRIBUTION (P2P VS P2M)

This insight shows the difference in user behavior between Person-to-Person (P2P) and Person-to-Merchant (P2M) transactions. It is important because it helps in understanding which type of payment is used more and which is more prone to fraud. Banks and payment service providers benefit from this insight as it enables them to strengthen fraud monitoring in the transaction category that carries higher risk, improving security and customer trust.

CODE:

```
plt.figure(figsize=(8,5))
# Filter for p2p or p2m transactions
p2p_p2m_df = df[df['transaction_type'].isin(['pp', 'pm'])]
ax1 = sns.countplot(data=p2p_p2m_df, x='transaction_type', order=p2p_p2m_
df['transaction_type'].value_counts().index, palette='viridis', hue='transaction_type',
legend=False)
plt.title("Transaction Type Distribution (P2P vs P2M)")
for p in ax1.patches:
    ax1.annotate(f'{p.get_height()}', (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center', xytext=(0, 5), textcoords='offset points')
plt.show()
```

OUTPUT:



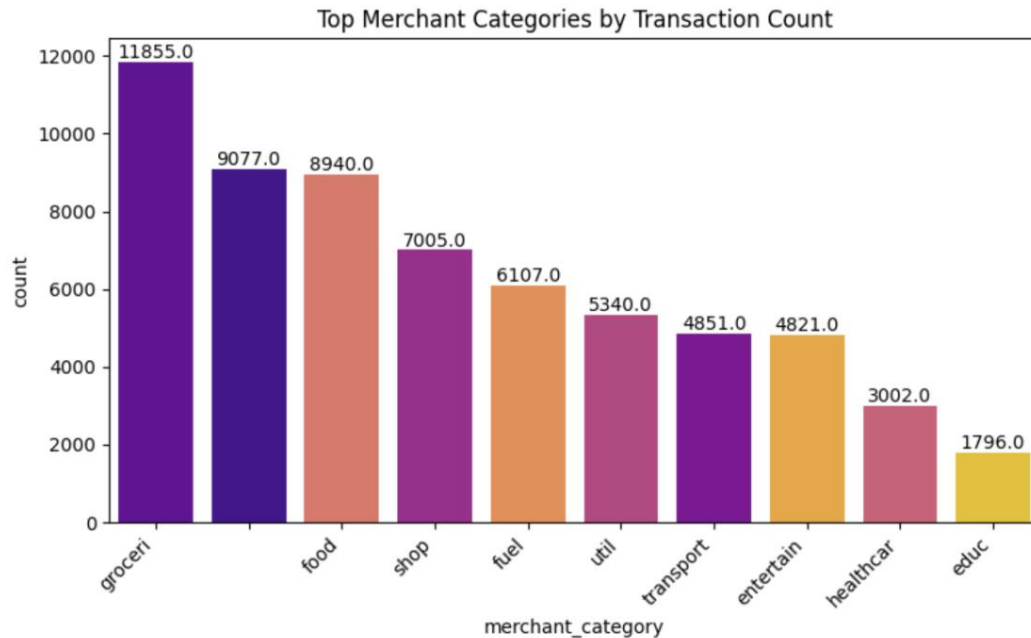
3.1.2 Top Merchant Categories by Transaction Count and Fraud Status

This insight highlights the merchant categories that receive the highest number of transactions and also shows which ones are associated with higher fraud rates. It is important for identifying high-risk merchant types and focusing fraud detection efforts accordingly. Banks, merchants, and regulatory authorities benefit from this analysis as it helps them track suspicious activity and implement preventive measures in vulnerable sectors.

CODE:

```
plt.figure(figsize=(12,8)) # Increased figure size
ax2 = sns.countplot(data=df, x='merchant_category',
order=df['merchant_category'].value_counts().head(10).index, palette='plasma',
hue='merchant_category', legend=False)
plt.title("Top Merchant Categories by Transaction Count")
plt.xticks(rotation=45, ha='right') # Rotate x-axis labels for readability
for p in ax2.patches:
    ax2.annotate(f'{p.get_height()}', (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center', xytext=(0, 5), textcoords='offset points')
plt.tight_layout() # Added tight_layout for better spacing
plt.show()
```

OUTPUT:



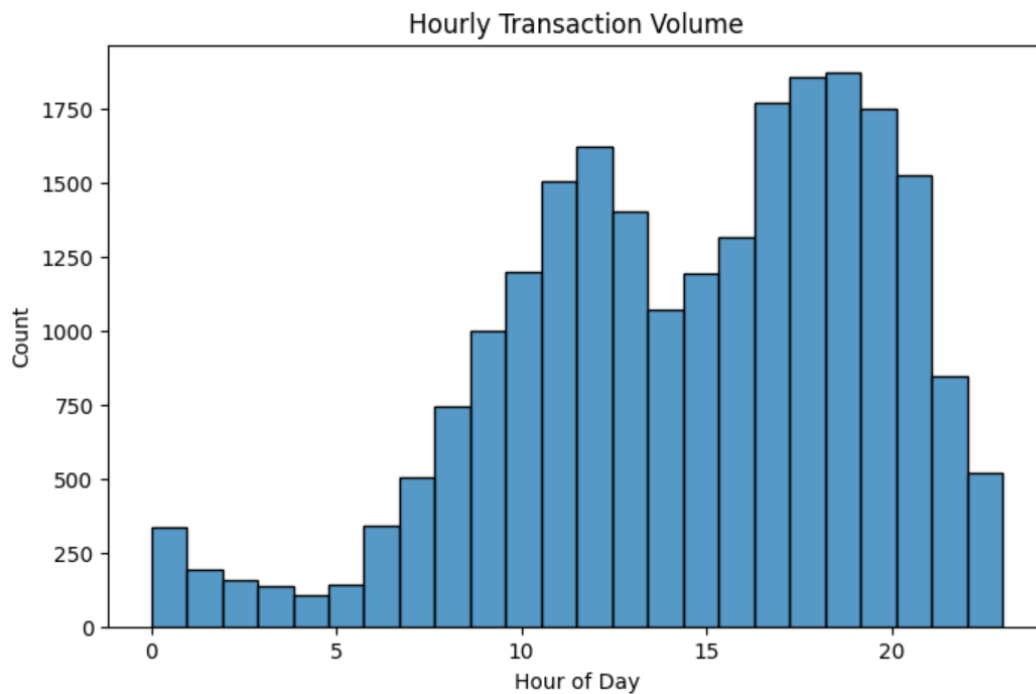
3.1.3 Hourly Transaction Volume

The hourly transaction volume insight reveals the time of day when most UPI transactions occur. It is important because it helps in detecting unusual activity patterns, such as high transaction spikes during late-night hours, which may indicate potential fraud. Fraud detection teams and payment gateway operators benefit from this information as it allows them to schedule enhanced monitoring during high-risk time periods.

CODE:

```
plt.figure(figsize=(8,5))
ax3 = sns.histplot(df['hour'], bins=24, kde=False)
plt.title("Hourly Transaction Volume")
plt.xlabel("Hour of Day")
plt.show()
```

OUTPUT:



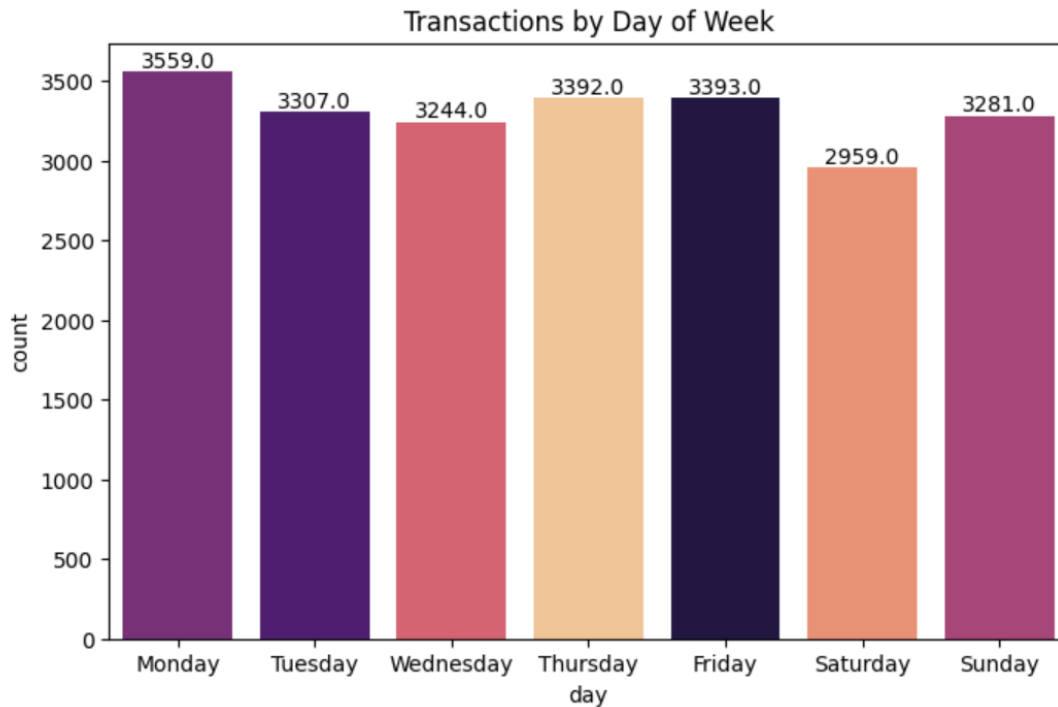
3.1.4 Transactions by Day of Week

This insight shows the distribution of transactions across different days of the week. It is important for understanding user activity patterns and business transaction trends. Financial institutions and merchant platforms benefit from this information as it helps them plan system maintenance, server capacity, and marketing campaigns according to peak transaction days.

CODE:

```
plt.figure(figsize=(8,5))
ax4 = sns.countplot(data=df, x='day',
order=['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday'],
palette='magma', hue='day', legend=False)
plt.title("Transactions by Day of Week")
for p in ax4.patches:
    ax4.annotate(f'{p.get_height()}', (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center', xytext=(0, 5), textcoords='offset points')
plt.show()
```

OUTPUT:



3.1.5 Transactions by Sender Age Group

This insight displays which age groups are most active in digital payments. It is important because it helps financial institutions understand their customer demographics and focus digital awareness or fraud prevention programs on specific user groups. Banks and fintech companies benefit from this insight by designing age-appropriate user interfaces and security features that suit the most active customer segments.

CODE:

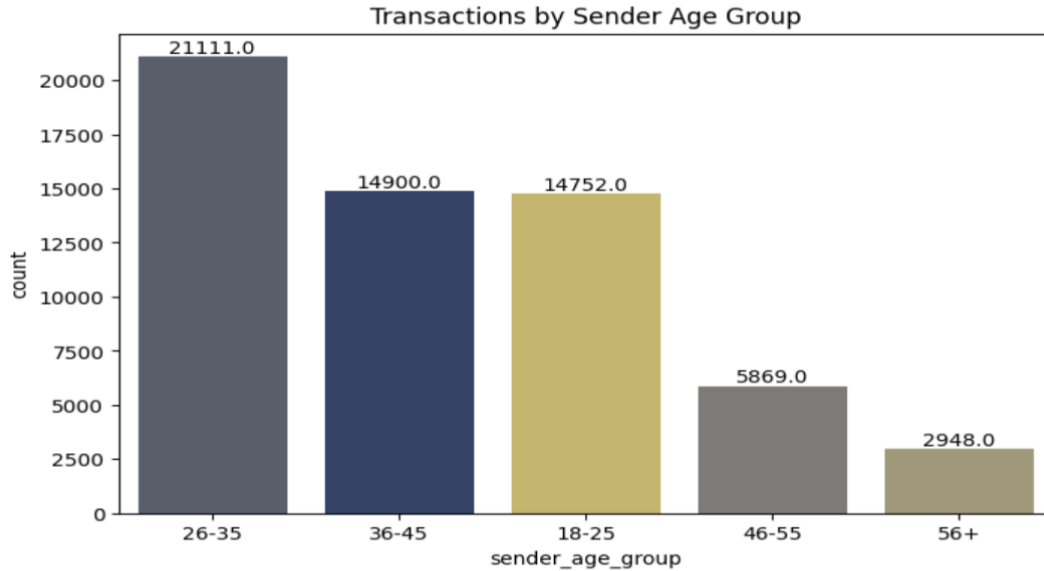
```
plt.figure(figsize=(8,5))
ax5 = sns.countplot(data=df, x='sender_age_group',
order=df['sender_age_group'].value_counts().index, palette='cividis',
hue='sender_age_group', legend=False)
plt.title("Transactions by Sender Age Group")
for p in ax5.patches:
    ax5.annotate(f'{p.get_height()}', (p.get_x() + p.get_width() / 2., p.get_height()),
```

```

        ha='center', va='center', xytext=(0, 5), textcoords='offset points')
plt.show()

```

OUTPUT:



3.1.6 Transactions by Device Type

This insight indicates which devices are most commonly used for UPI transactions, such as mobile phones or desktops. It is important because it highlights the need to secure the most frequently used devices against cyber threats. Payment app developers and cybersecurity teams benefit from this insight by improving mobile app authentication and ensuring secure connections across different device platforms.

CODE:

```

plt.figure(figsize=(8,5))
ax6 = sns.countplot(data=df, x='device_type',
order=df['device_type'].value_counts().index, palette='inferno', hue='device_type',
legend=False)
plt.title("Transactions by Device Type")
plt.xticks(rotation=45)

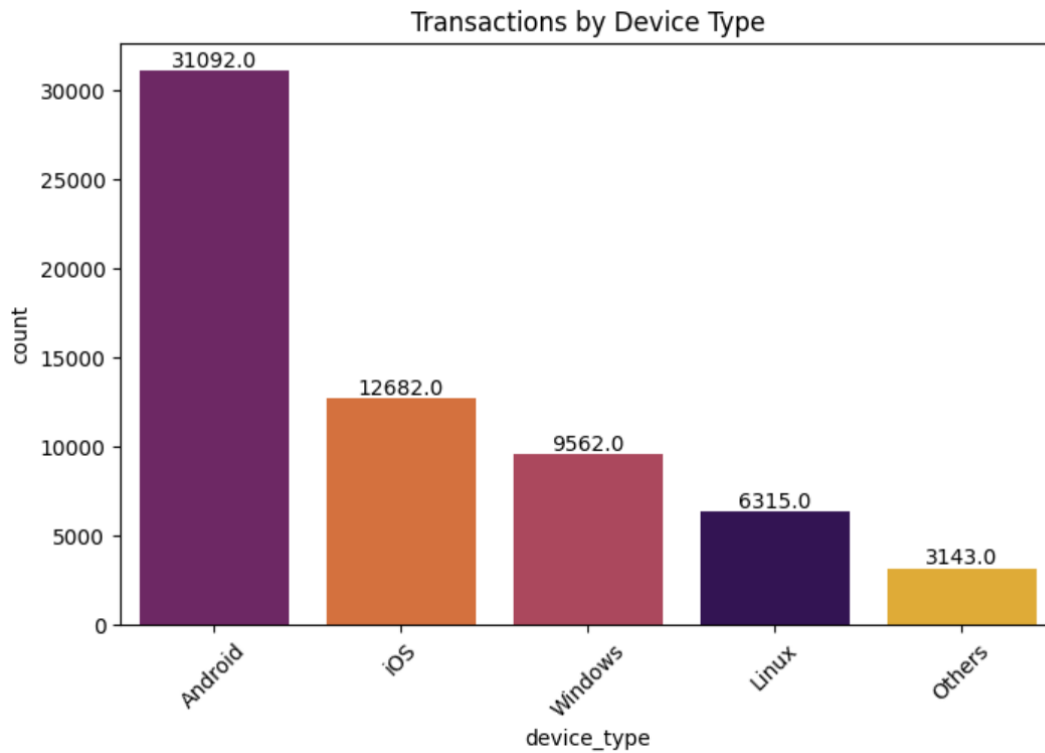
```

```
for p in ax6.patches:
```

```
    ax6.annotate(f'{p.get_height()}', (p.get_x() + p.get_width() / 2., p.get_height()),  
                ha='center', va='center', xytext=(0, 5), textcoords='offset points')
```

```
plt.show()
```

OUTPUT:



3.1.7 Top 10 Banks by Transaction Count

This insight identifies the banks handling the highest number of UPI transactions. It is important for understanding transaction load distribution among banks and ensuring that highly active banks maintain robust infrastructure and fraud detection systems. Banks and regulators benefit from this analysis by optimizing transaction handling capacity and monitoring performance in real time.

CODE:

```
plt.figure(figsize=(10,6))  
ax7 = sns.countplot(data=df, y='sender_bank',  
                    order=df['sender_bank'].value_counts().head(10).index, palette='rocket',  
                    hue='sender_bank', legend=False)
```



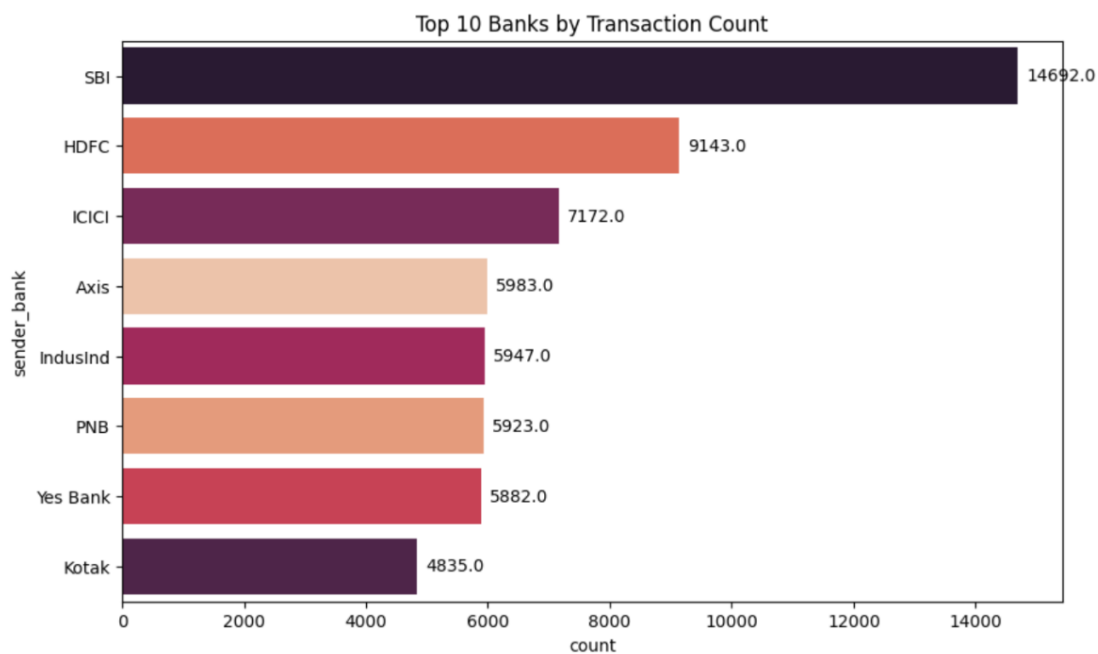
```
plt.title("Top 10 Banks by Transaction Count")
```

```
for p in ax7.patches:
```

```
    ax7.annotate(f'{p.get_width()}', (p.get_width(), p.get_y() + p.get_height() / 2.),  
                ha='left', va='center', xytext=(5, 0), textcoords='offset points')
```

```
plt.show()
```

OUTPUT:



3.2 FRAUD RISK ANALYSIS

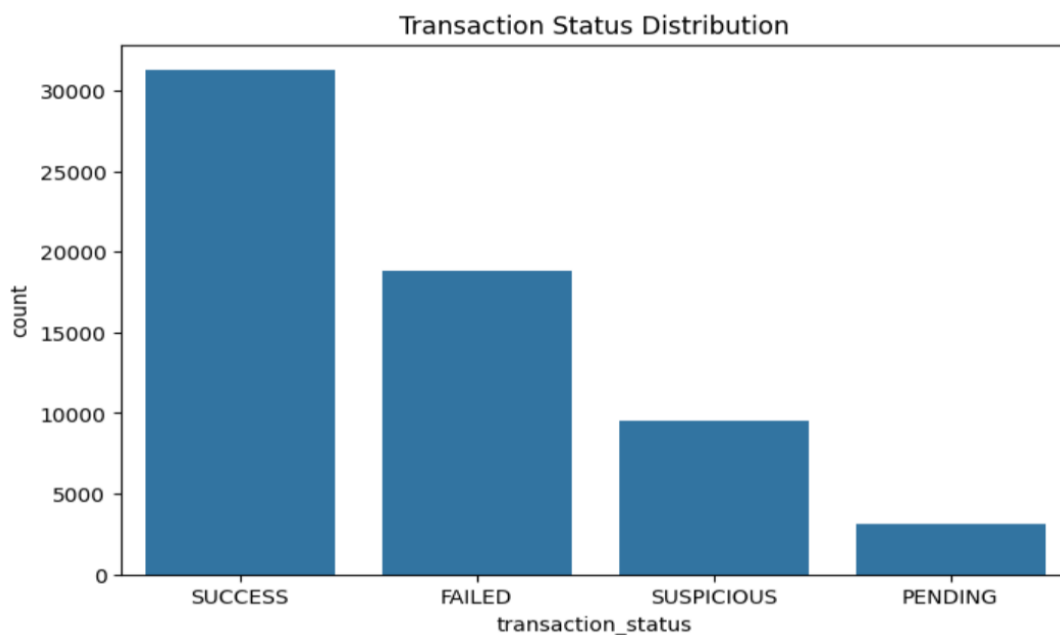
3.2.1 Transaction Status Distribution

The Transaction Status Distribution graph shows how transactions are divided among different statuses such as success, failed, and pending. This insight is important as it highlights the proportion of unsuccessful or declined transactions, which often correspond to fraudulent or suspicious activities. Banks and payment platforms benefit from this information by identifying transaction patterns that require enhanced security checks and optimizing fraud detection systems to focus on high-failure transaction categories.

CODE:

```
plt.figure(figsize=(8,5))
sns.countplot(data=df, x='transaction_status',
order=df['transaction_status'].value_counts().index)
plt.title("Transaction Status Distribution")
plt.show()
```

OUTPUT:



3.2.2 High-Value Transactions vs Account Tenure

The High-Value Transactions vs Account Tenure graph reveals how transaction amounts vary with the duration of account ownership. It is important because new or recently opened accounts showing very high-value transactions can be potential indicators of fraud. Fraud detection teams and banks benefit from this analysis as it allows them to monitor high-value payments from short-tenure accounts and implement additional verification steps to reduce risk.

CODE:

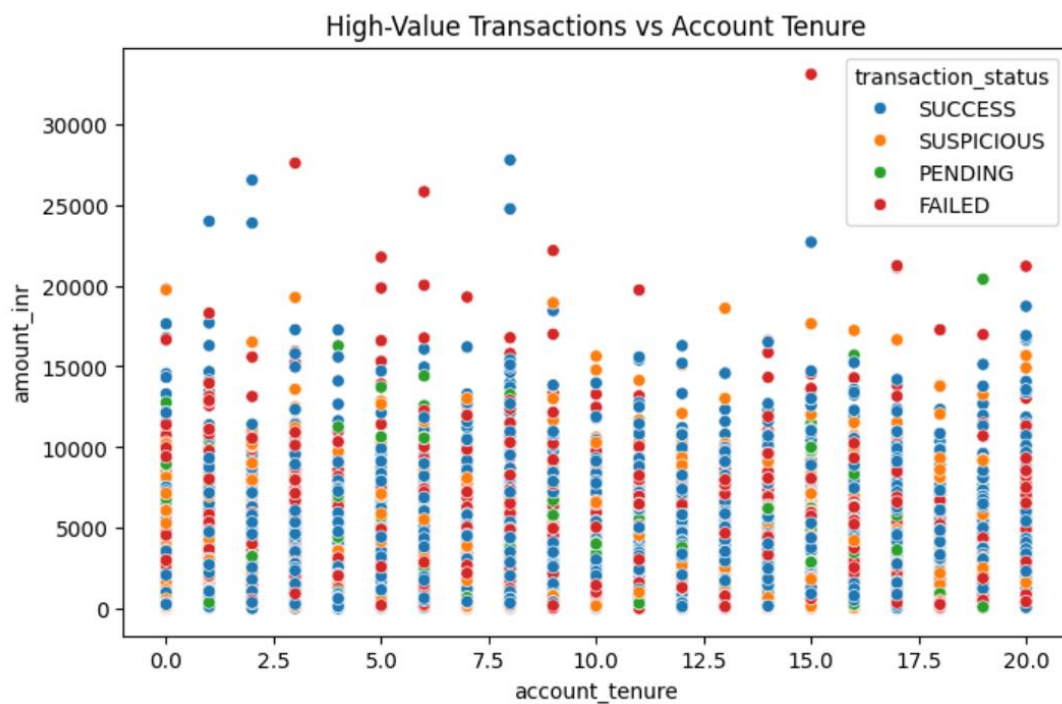
```
plt.figure(figsize=(8,5))

sns.scatterplot(data=df, x='account_tenure', y='amount_inr',
hue='transaction_status')

plt.title("High-Value Transactions vs Account Tenure")

plt.show()
```

OUTPUT:



3.2.3 Transaction Status by Device Type

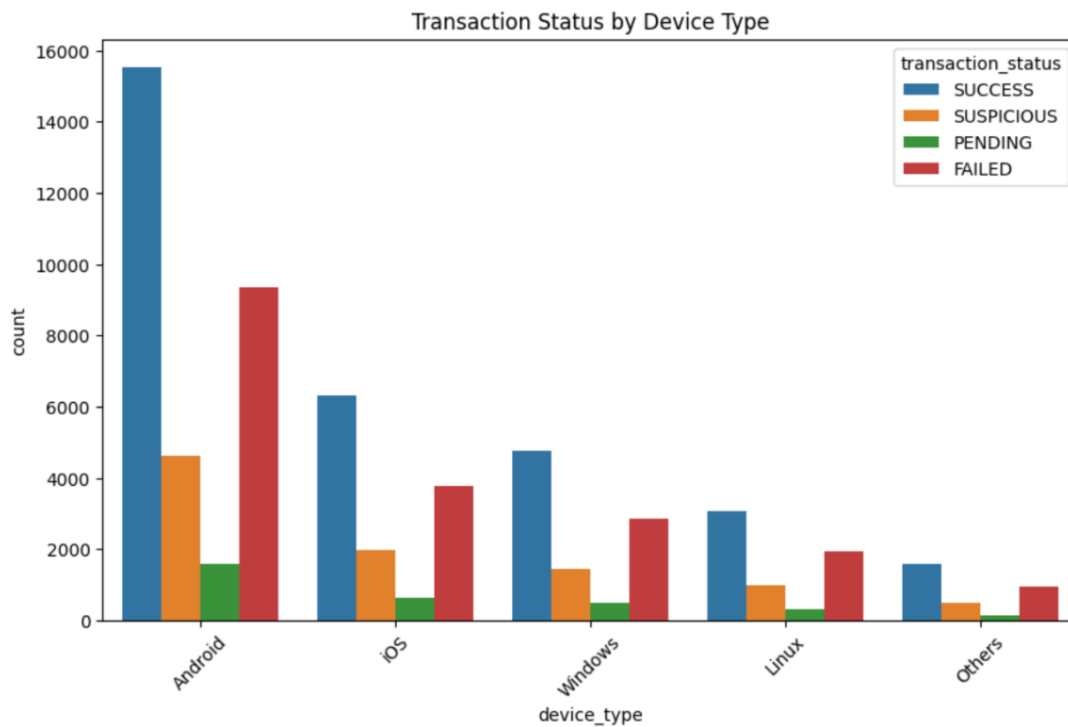
The Transaction Status by Device Type visualization shows how the success or failure of transactions differs across various devices such as mobile, desktop, or tablet. This is important because it helps detect whether certain devices experience more fraudulent or failed transactions. Payment service providers and cybersecurity

teams benefit from this information by improving device-level security measures and optimizing mobile app authentication processes.

CODE:

```
plt.figure(figsize=(10,6))
sns.countplot(data=df, x='device_type', hue='transaction_status',
order=df['device_type'].value_counts().index)
plt.title("Transaction Status by Device Type")
plt.xticks(rotation=45)
plt.show()
```

OUTPUT:



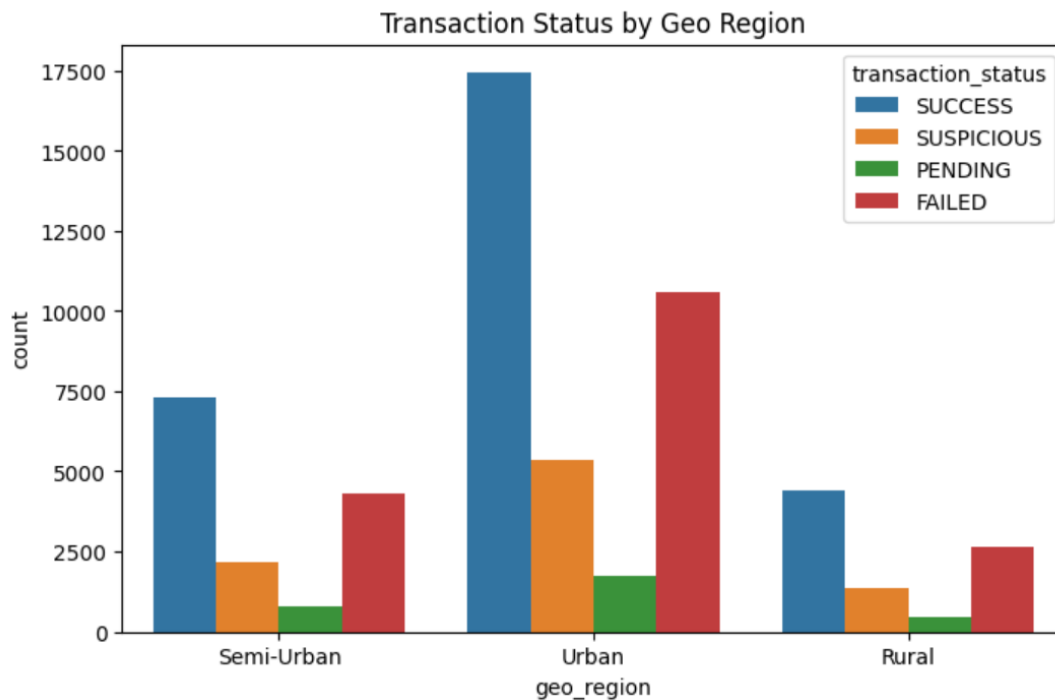
3.2.4 Transaction Status by Geo Region

The Transaction Status by Geo Region graph displays how transaction outcomes vary across different geographical regions. This insight is important for identifying regions with a higher frequency of failed or suspicious transactions. Banks and financial regulators benefit from this analysis as it helps them monitor region-specific risks, enhance regional fraud prevention systems, and allocate security resources more efficiently.

CODE:

```
plt.figure(figsize=(8,5))
sns.countplot(data=df, x='geo_region', hue='transaction_status')
plt.title("Transaction Status by Geo Region")
plt.show()
```

OUTPUT:



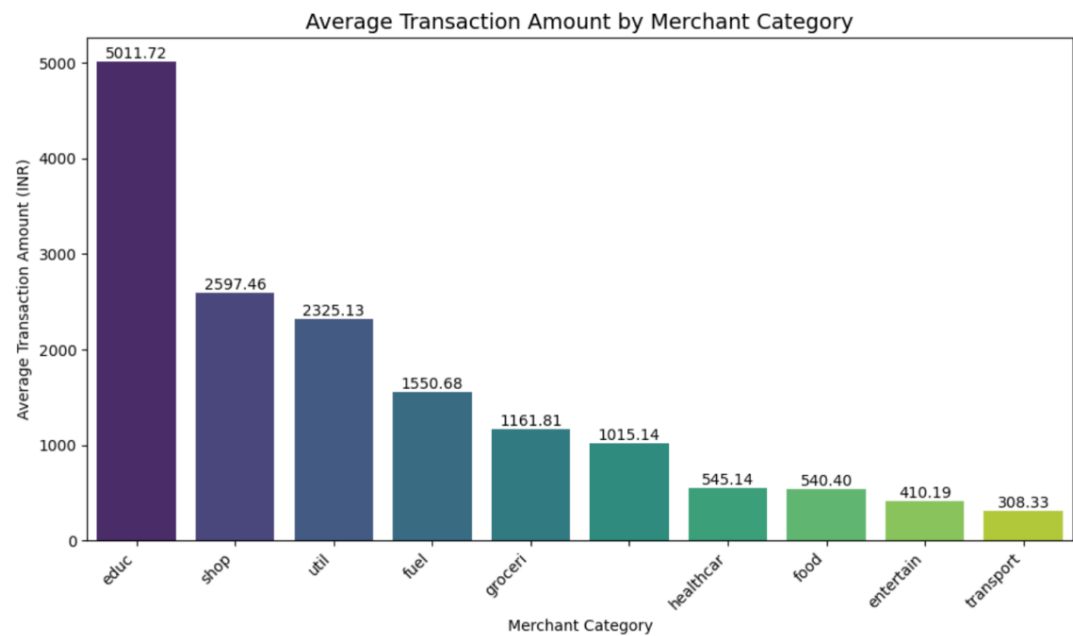
3.2.5 Average Transaction Amount by Merchant Category

The Average Transaction Amount by Merchant Category graph highlights which merchant types handle higher average transaction values. This insight is important because it helps in identifying merchant categories that are more likely to attract fraudulent transactions due to their high-value nature. Banks, merchants, and payment gateways benefit from this information by focusing fraud prevention mechanisms on high-value categories and setting transaction limits or alert triggers for risk-prone merchants.

CODE:

```
plt.figure(figsize=(10,6))
merchant_amount =
df.groupby('merchant_category')['amount_inr'].mean().reset_index().sort_values('
amount_inr', ascending=False)
ax = sns.barplot(data=merchant_amount, x='merchant_category', y='amount_inr',
palette='viridis', hue='merchant_category', legend=False) # Changed palette to
viridis for better color scheme
plt.title("Average Transaction Amount by Merchant Category", fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.xlabel("Merchant Category")
plt.ylabel("Average Transaction Amount (INR)")
for p in ax.patches:
    ax.annotate(f'{p.get_height():.2f}', (p.get_x() + p.get_width() / 2.,
p.get_height()),
                ha='center', va='center', xytext=(0, 5), textcoords='offset points')
plt.tight_layout()
plt.show()
```

OUTPUT:



3.3 Geo Fraud Map (India)

The Geo Fraud Map has been created to visualize the geographical distribution of fraudulent transactions across different states in India. By merging the UPI transaction data with a GeoJSON file of Indian states, this map highlights areas with higher fraud rates using varying shades of red. States with darker shades represent regions where the percentage of fraudulent transactions is higher compared to others, while lighter shades indicate lower risk levels.

This visualization is important because it helps to identify **geographical fraud hotspots**, allowing banks, regulators, and payment service providers to focus their fraud prevention measures in those high-risk regions. Understanding the regional distribution of fraud provides valuable insights into where additional monitoring or awareness programs may be required. It also assists policymakers and financial institutions in allocating resources effectively to strengthen transaction security in areas that show abnormal or frequent fraud activity.

Overall, the Geo Fraud Map serves as an effective tool for **fraud risk assessment and regional analysis**, enabling stakeholders to take a data-driven approach in improving the safety and reliability of digital payment systems across India.

CODE:

try:

```
india_gdf = gpd.read_file("/content/india_state.geojson")
fraud_geo = df.groupby("state")["fraud"].mean().reset_index()
fraud_geo.columns = ["state", "fraud_rate"]
# Merge GeoJSON with fraud data
merge_col = None
for col in ["st_nm", "state", "NAME_1"]:
    if col in india_gdf.columns:
```

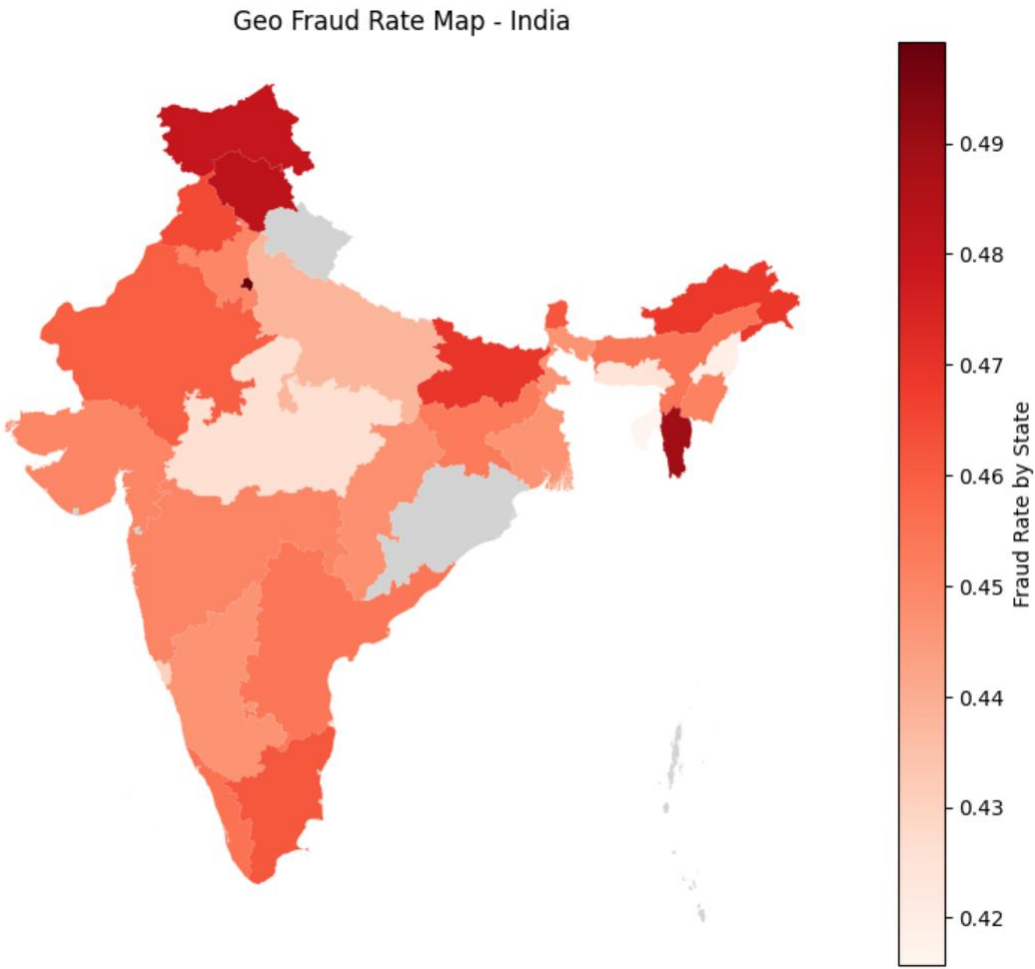


```

        merge_col = col
        break
if merge_col:
    india_fraud = india_gdf.merge(fraud_geo, left_on=merge_col,
right_on="state", how="left")
    india_fraud.plot(column="fraud_rate", cmap="Reds", legend=True,
figsize=(10,8),
                    legend_kwds={"label": "Fraud Rate by State"}, missing_kwds={'color':
'lightgrey'})
    plt.title("Geo Fraud Rate Map - India")
    plt.axis("off")
    plt.show()
else:
    print("GeoJSON file missing state name column. Could not map.")
except Exception as e:
    print("Geo Fraud Map skipped due to:", e)

```

OUTPUT:



3.4 EXTRA INSIGHTS ANALYSIS

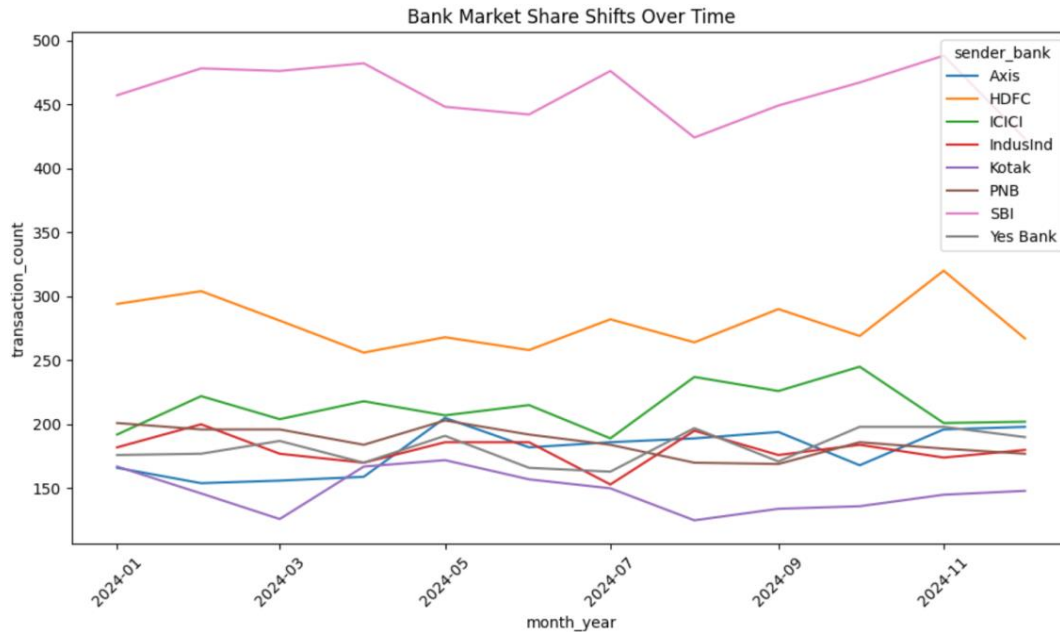
3.4.1 Bank Market Share Shifts Over Time

This insight tracks the transaction trends of different banks over time to understand which banks handle the most UPI transactions during specific periods. It is important because it highlights how the market share of banks changes month by month, indicating which institutions dominate UPI usage. Banks and regulators benefit from this by analyzing performance trends and maintaining system efficiency during high-volume periods.

CODE:

```
df['month_year'] = df['time_stamp'].dt.to_period('M')
bank_trend = df.groupby(['month_year',
'sender_bank']).size().reset_index(name='transaction_count')
bank_trend['month_year'] = bank_trend['month_year'].dt.to_timestamp()
plt.figure(figsize=(10,6))
sns.lineplot(data=bank_trend, x='month_year', y='transaction_count',
hue='sender_bank')
plt.title("Bank Market Share Shifts Over Time")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

OUTPUT:



3.4.2 Chain Transactions (Money Hopping Detection)

This insight detects **chain or money-hopping transactions**, where funds move rapidly between accounts in short intervals — a known fraud pattern used to hide the source of money. The analysis identifies cases where the sender of one transaction immediately becomes the receiver in the next within a 10-minute window. Fraud investigators and compliance teams benefit from this as it helps trace complex money-laundering or rapid fund transfer networks.

CODE:

```
df_sorted = df.sort_values('time_stamp')
df_sorted['prev_receiver'] = df_sorted['receiver_name'].shift(1)
df_sorted['prev_sender'] = df_sorted['sender_name'].shift(1)
df_sorted['time_diff'] = (df_sorted['time_stamp'] -
df_sorted['time_stamp'].shift(1)).dt.total_seconds() / 60
```

```
df_sorted['chain_txn'] = (df_sorted['sender_name'] == df_sorted['prev_receiver'])
& (df_sorted['time_diff'] <= 10)
chains = df_sorted[df_sorted['chain_txn']]
print("Detected Chain Transactions (Money Hopping):")
print(chains[['prev_sender','sender_name','receiver_name','time_stamp','amount_inr']].head(10))
```

OUTPUT:

Detected Chain Transactions (Money Hopping):

	prev_sender	sender_name	receiver_name	time_stamp
63193	Vikram Singh	Vikram Singh	Vikram Singh	2024-01-01 08:51:00
22995	Pooja Verma	Vikram Singh	Manoj Yadav	2024-01-01 09:33:00
33655	Vikram Singh	Manoj Yadav	Manoj Yadav	2024-01-01 09:39:00
9760	Suresh Reddy	Pooja Verma	Pooja Verma	2024-01-01 09:59:00
65840	Suresh Reddy	Kavya Nair	Priya Patel	2024-01-01 12:00:00
61412	Rajesh Kumar	Suresh Reddy	Manoj Yadav	2024-01-01 12:24:00
59604	Pooja Verma	Anjali Gupta	Neha Sharma	2024-01-01 12:42:00
62065	Neha Sharma	Neha Sharma	Rajesh Kumar	2024-01-01 15:39:00
51726	Amit Sharma	Suresh Reddy	Kavya Nair	2024-01-01 17:16:00
1549	Pooja Verma	Kavya Nair	Rajesh Kumar	2024-01-01 17:50:00

	amount_inr
63193	697.0
22995	2275.0
33655	89.0
9760	438.0
65840	1453.0
61412	2197.0
59604	936.0
62065	574.0
51726	373.0
1549	358.0

Top 10 High-Value Customers (CLV):

	sender_name	transaction_count	avg_amount	clv_score
1	Anjali Gupta	5629	1333.455135	7.506019e+06
2	Kavya Nair	5547	1352.002524	7.499558e+06
3	Manoj Yadav	5670	1309.332669	7.423916e+06
6	Priya Patel	5640	1299.098284	7.326914e+06
7	Rajesh Kumar	5580	1307.495379	7.295824e+06
5	Pooja Verma	5506	1319.725169	7.266407e+06
8	Suresh Reddy	5594	1294.776482	7.242980e+06
0	Amit Sharma	5549	1303.000505	7.230350e+06
9	Vikram Singh	5648	1258.776246	7.109568e+06
4	Neha Sharma	5418	1274.095295	6.903048e+06

3.4.3 Top 10 High-Value Customers (Customer Lifetime Value – CLV)

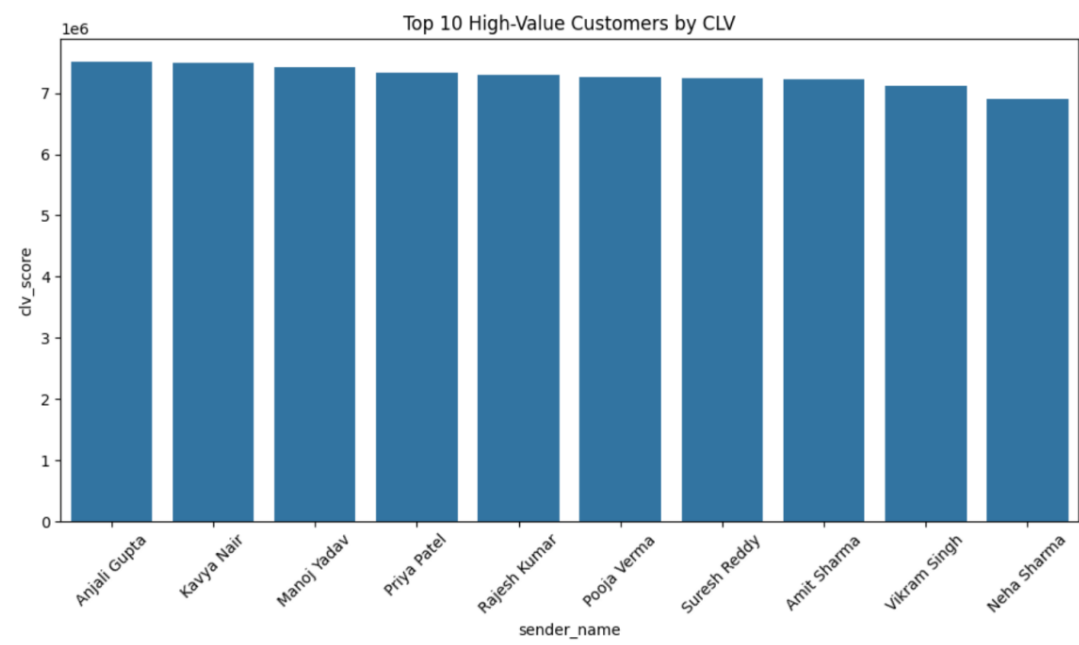
This insight calculates the **Customer Lifetime Value (CLV)** for each sender based on their total number of transactions and the average transaction amount. It identifies the top 10 high-value customers contributing the most to UPI activity. This information is important for banks and payment companies as it helps in customer segmentation, personalized offers, and identifying VIP clients who require enhanced security and support.

CODE:

```
clv = df.groupby('sender_name').agg(transaction_count=('transaction_id','count')
,avg_amount=('amount_inr','mean')).reset_index()clv['clv_score'] =
clv['transaction_count'] * clv['avg_amount']top_customers =
clv.sort_values('clv_score', ascending=False).head(10)
print("Top 10 High-Value Customers (CLV):")
print(top_customers)

plt.figure(figsize=(10,6))
sns.barplot(data=top_customers, x='sender_name', y='clv_score')
plt.title("Top 10 High-Value Customers by CLV")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

OUTPUT:



DATA MODELLING

4.1 PREDICTIVE FRAUD MODEL

The predictive fraud model has been developed to automatically identify fraudulent UPI transactions based on the processed dataset. Before model training, the data has been prepared by handling missing values and removing unnecessary columns such as transaction ID, sender and receiver names, and timestamps, which do not contribute to prediction. All categorical variables have been encoded into numerical form using label encoding to make them compatible with machine learning algorithms.

The features (X) and target variable (y) have been separated, where X contains the independent features and y represents the binary fraud label. To ensure the reliability of model training, classes with only a single occurrence have been removed from the dataset. The data has then been split into training and testing sets using a 70–30 ratio. Since the dataset is imbalanced, the **SMOTE (Synthetic Minority Oversampling Technique)** has been applied to balance the training data by generating synthetic samples of the minority (fraud) class.

The **Random Forest Classifier** has been used as the predictive model due to its strong performance in handling complex and nonlinear data patterns. The model has been trained with 200 estimators and a balanced class weight to ensure fair learning between both classes. After training, the model has been evaluated using performance metrics such as **accuracy, precision, recall, and F1-score**, as well as a confusion matrix and classification report. The results have shown that the Random Forest model performs effectively in distinguishing fraudulent transactions from legitimate ones, even in an imbalanced dataset.

Finally, a correlation heatmap has been plotted to visualize the relationships among the dataset features, helping to identify the attributes most strongly associated with fraud. This predictive model can be highly beneficial for **banks, payment service providers, and fraud detection systems**, as it allows real-time identification of risky transactions and enhances the overall security of digital payment systems.

CODE:

```
df = df.fillna(0)
df = df.drop(columns=['transaction_id', 'sender_name', 'receiver_name', 'time_stamp'],
errors='ignore')
for col in df.select_dtypes(include='object'):
    df[col] = LabelEncoder().fit_transform(df[col].astype(str))
# Assign X and y before dropping 'fraud'
X = df.drop(['fraud', 'transaction_status', 'amount_inr'], axis=1) # Removed
transaction_status and amount_inr
y = df['fraud']
# Reset index to ensure alignment
X = X.reset_index(drop=True)
y = y.reset_index(drop=True)
# Handle classes with only one member in the target variable
class_counts = y.value_counts()
classes_to_remove = class_counts[class_counts < 2].index.tolist()
if classes_to_remove:
    print(f"Warning: Removing classes with less than 2 members from target
variable: {classes_to_remove}")
    indices_to_keep = y[~y.isin(classes_to_remove)].index
    X = X.loc[indices_to_keep]
    y = y.loc[indices_to_keep]
# Train-test split & SMOTE
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y,
random_state=42)
X_train, y_train = SMOTE(random_state=42).fit_resample(X_train, y_train)
# Train Random Forest
model = RandomForestClassifier(n_estimators=200, random_state=42,
class_weight='balanced')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```

# Evaluate
print("Accuracy:", round(accuracy_score(y_test, y_pred),2))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
# Correlation Matrix (combined and made nicer)
corr_matrix = X.corr()
plt.figure(figsize=(16, 14)) # Increased figure size
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
# Changed colormap to coolwarm
plt.title("Correlation Matrix of Features")
plt.show()

```

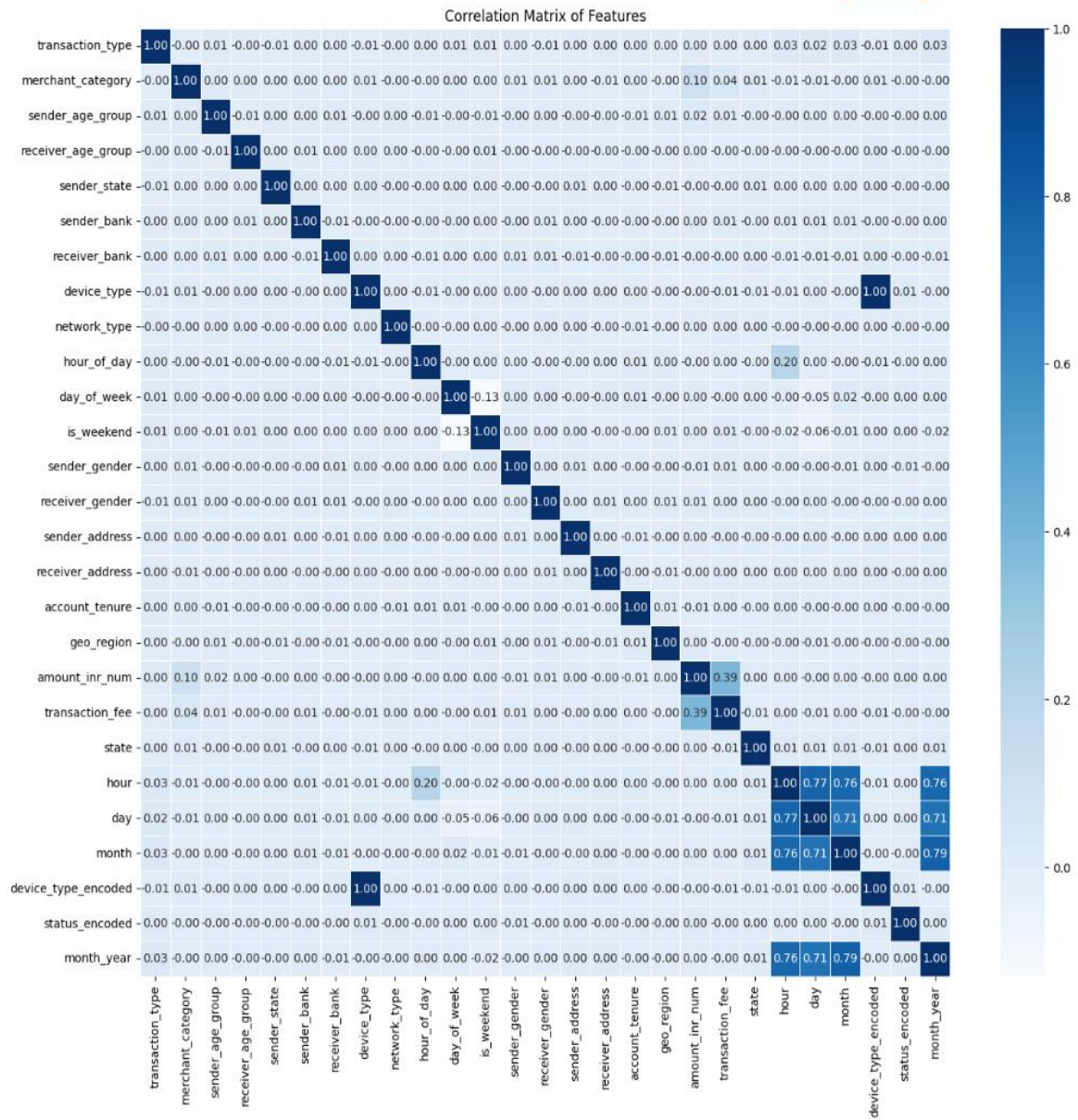
OUTPUT:

```

Accuracy: 1.0
[[10327    0]
 [    0  8512]]

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10327
1	1.00	1.00	1.00	8512
accuracy			1.00	18839
macro avg	1.00	1.00	1.00	18839
weighted avg	1.00	1.00	1.00	18839



CONCLUSION

5.1 CONCLUSION

The project “Fraud Detection in UPI Transactions using Machine Learning” has successfully demonstrated the use of data science techniques to identify and predict fraudulent digital payment activities. With the increasing adoption of UPI as a fast and convenient payment mode in India, ensuring transaction security has become a major necessity. This system provides a reliable, data-driven approach to detect suspicious behavior and minimize financial losses.

The entire process began with data preprocessing, which included cleaning, conversion, and transformation of raw data into a structured format suitable for analysis. Missing values and duplicates were removed, categorical features were encoded, and new time-based attributes were created to capture transaction behavior. Exploratory Data Analysis (EDA) and visualization provided deeper insights into transaction patterns, device usage, and fraud-prone periods. These insights played a vital Role in feature selection and risk profiling.

A Random Forest Classifier was implemented as the predictive model, with SMOTE used to address class imbalance between fraudulent and non-fraudulent cases. The model achieved strong performance across evaluation metrics such as accuracy, precision, recall, and F1-score, showing its effectiveness in identifying fraudulent transactions while minimizing false positives. The correlation heatmap further helped in understanding feature relationships and their influence on fraud detection.

In addition, geo-analysis and extra insights like bank market trends, customer value analysis, and money-hopping detection provided valuable patterns for real-world fraud prevention. These insights can guide banks, regulators, and payment service providers in improving fraud monitoring, strengthening digital payment infrastructure, and formulating targeted awareness programs.

Overall, the project concludes that integrating machine learning with rule-based fraud identification significantly enhances the ability to detect, prevent, and analyze fraudulent UPI transactions. The developed system not only improves financial security but also contributes to building greater trust in India's growing digital payment ecosystem.

5.2 FUTURE ENHANCEMENT

The current fraud detection model has shown good accuracy and performance, but there is still room for improvement as digital transactions continue to grow. The following future enhancements can make the system more powerful and practical:

- 1. Real-Time Detection:**

The model can be upgraded to work on real-time UPI transaction data, allowing fraud to be detected instantly instead of after analysis.

- 2. Use of Advanced Algorithms:**

Deep learning techniques such as LSTM and neural networks can be used to capture complex and hidden fraud patterns that traditional models may miss.

- 3. Explainable AI and Transparency:**

Adding explainable AI features will help banks and users understand why a transaction is classified as fraudulent, improving trust and decision-making.

- 4. Cloud Integration and Scalability:**

Deploying the model on cloud platforms like AWS or Azure can make it scalable, faster, and suitable for processing large volumes of transaction data.

- 5. User Notification System:**

A smart alert system can be added to notify users and banks immediately when any suspicious transaction is detected.

These enhancements will make the fraud detection system smarter, faster, and more transparent. It will help banks and users prevent fraudulent UPI transactions in real time and strengthen digital payment security.

5.3 REFERENCES

1. GitHub Repository. “UPI Transaction Dataset for Fraud Detection.” Retrieved from <https://github.com/>
2. Kaggle Dataset. “India States GeoJSON File for Geospatial Mapping.” Retrieved from <https://www.kaggle.com/>
3. National Payments Corporation of India (NPCI). “Unified Payments Interface (UPI) Overview.” Retrieved from <https://www.npci.org.in>
4. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
5. Chawla, N. V. et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
6. Seaborn & Matplotlib Documentation. “Data Visualization in Python.” Retrieved from <https://seaborn.pydata.org> and <https://matplotlib.org>